

Abstract Class

The Rules of Abstract Methods

1. **No Body:** Abstract methods end with a semicolon ; instead of curly braces {}.
2. **The abstract Keyword:** You must place the word abstract before the method name.
3. **Abstract Class Requirement:** If a class contains even *one* abstract method, **the entire class must be declared as abstract**. You cannot instantiate (create a new) abstract class directly.
4. **Child Obligation:** Any normal child class that inherits from an abstract parent **must override and write the body** for all abstract methods, or your code will fail to compile.

The Abstract Parent Class

Traditional gas car accelerates by burning fuel, while an electric car accelerates by pulling power from a battery pack. Instead of writing a generic, useless default method in the parent class, we make it **abstract**.

```
// We add 'abstract' to the class because it contains an abstract method
public abstract class Vehicle {
    public String color;

    public Vehicle(String color) {
        this.color = color;
    }

    // Standard method: Inherited normally by all children
    public void honk() {
        System.out.println("Beep beep!");
    }

    // ABSTRACT METHOD: No curly braces! Just a rule for the children.
    public abstract void accelerate();
}
```

The Child Classes (Fulfilling the Contract)

Now, any child class that extends Vehicle is forced by Java to implement accelerate(). If you forget to add it, Java will give you a compilation error.

```
public class GasCar extends Vehicle {
    public GasCar(String color) {
        super(color);
    }

    // We MUST override and provide a body for the abstract method
    @Override
    public void accelerate() {
        System.out.println("The gas engine revs! Fuel is injected into cylinders");
    }
}

public class ElectricCar extends Vehicle {
    public ElectricCar(String color) {
        super(color);
    }

    // We MUST override and provide a body for the abstract method too
    @Override
    public void accelerate() {
        System.out.println("The electric motor hums instantly! Battery power surges");
    }
}
```

Running the Code in **Main**

Remember, you **cannot** do `Vehicle v = new Vehicle("Red");` because abstract classes are incomplete blueprints. You can only instantiate the concrete children.

```
public class Main {  
    public static void main(String[] args) {  
  
        // Instantiate the concrete child classes  
        GasCar mustang = new GasCar("Blue");  
        ElectricCar tesla = new ElectricCar("White");  
  
        // Both can use the normal inherited method  
        mustang.honk(); // Prints: Beep beep!  
  
        // Each executes its own mandatory version of the abstract method  
        mustang.accelerate(); // Prints: The gas engine revs!...  
        tesla.accelerate();   // Prints: The electric motor hums!...  
    }  
}
```

Why Use Abstract Methods?

Abstract methods allow you to enforce a standard design structure across multiple different classes. It guarantees that no matter what kind of vehicle someone programs in the future, it is guaranteed to have an `accelerate()` action built into it.